# LA MAZMORRA

Modelos de Programación Orientada a Objetos (MPOO)

**Grupo:** 6
**Integrante:** Hernández Nieves Emily
**Número de cuenta:** 426058256

En esta práctica trabajamos con un videojuego llamado "La mazmorra", donde hay tres tipos de personajes: caballeros, magos y arqueros. Cada uno tiene diferente cantidad de vida y una forma distinta de atacar. También pueden utilizar herramientas, como espadas, bastones y arcos, que van perdiendo durabilidad cuando se usan.
Para realizar el programa se crearon las clases Herramienta, Inventario, Personaje, Caballero, Mago y Arquero, además de Main. Cada clase tiene una función diferente. Por ejemplo, Herramienta controla el daño y la durabilidad, Inventario permite guardar hasta tres herramientas y Personaje contiene las acciones generales, como atacar, recibir daño, curarse y ayudar a un compañero.

También se trabajó con el encapsulamiento, que sirve para proteger los datos de las clases y evitar que se puedan cambiar directamente sin respetar las reglas del programa. Al final se hicieron comprobaciones con los archivos de trampas y pruebas con JUnit para revisar que el código funcionara como se esperaba.

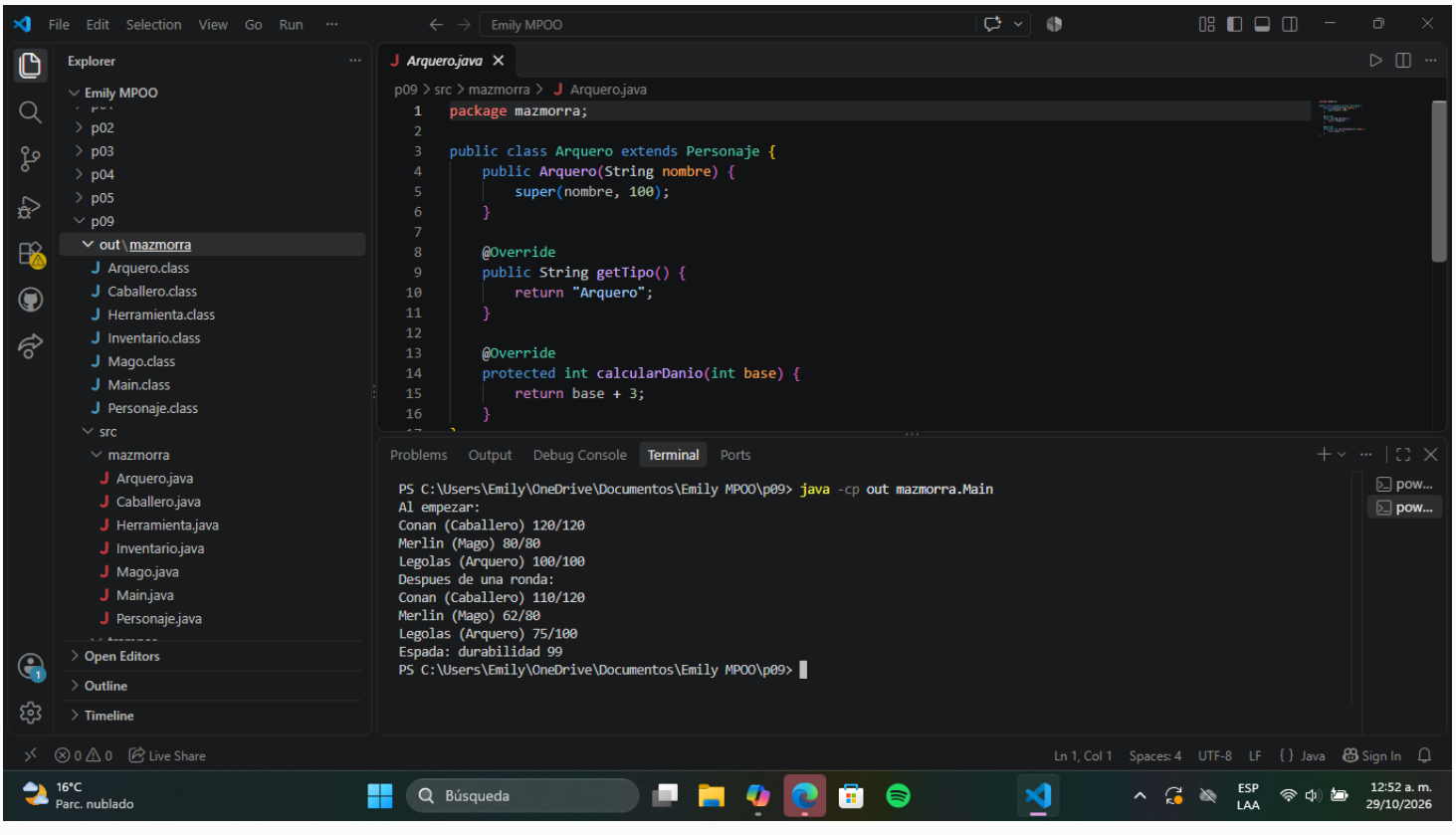



*Figura 1. Ejecución de Main.java*

Para revisar que el programa respetara sus reglas, se utilizaron dos archivos: Tramposo.java y TrampasDeAcceso.java. Primero se ejecutó Tramposo.java, que intenta realizar siete acciones que no deberían permitirse. Entre ellas están curarse usando una cantidad negativa de daño, superar la vida máxima, crear una herramienta con daño de 5000, agregar una cuarta herramienta, pedir una herramienta que no existe, atacar sin tener herramientas y curar a un personaje muerto.
El resultado esperado era que las siete trampas aparecieran como BLOQUEADA, lo que significa que el programa no permitió realizar esas acciones fuera de las reglas establecidas.

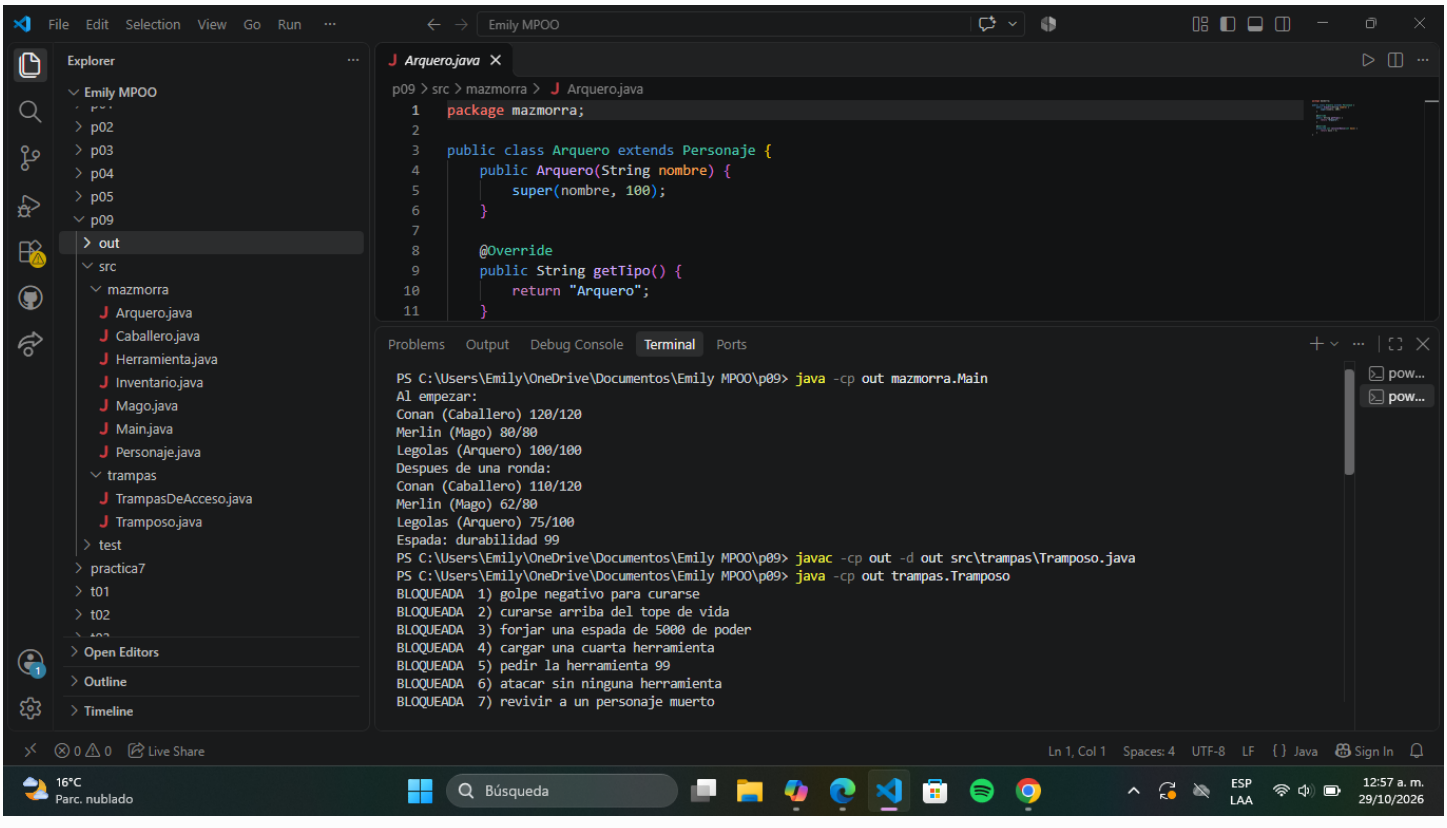



*Figura 2. Comprobación de las 7 trampas de reglas*

Después se intentó compilar TrampasDeAcceso.java. Este archivo intenta modificar directamente algunos atributos privados, como la vida del personaje, su inventario, la durabilidad de una herramienta y los datos internos del inventario. Al compilarlo aparecieron cinco errores que indicaban que no se podía acceder a esos atributos porque eran privados.

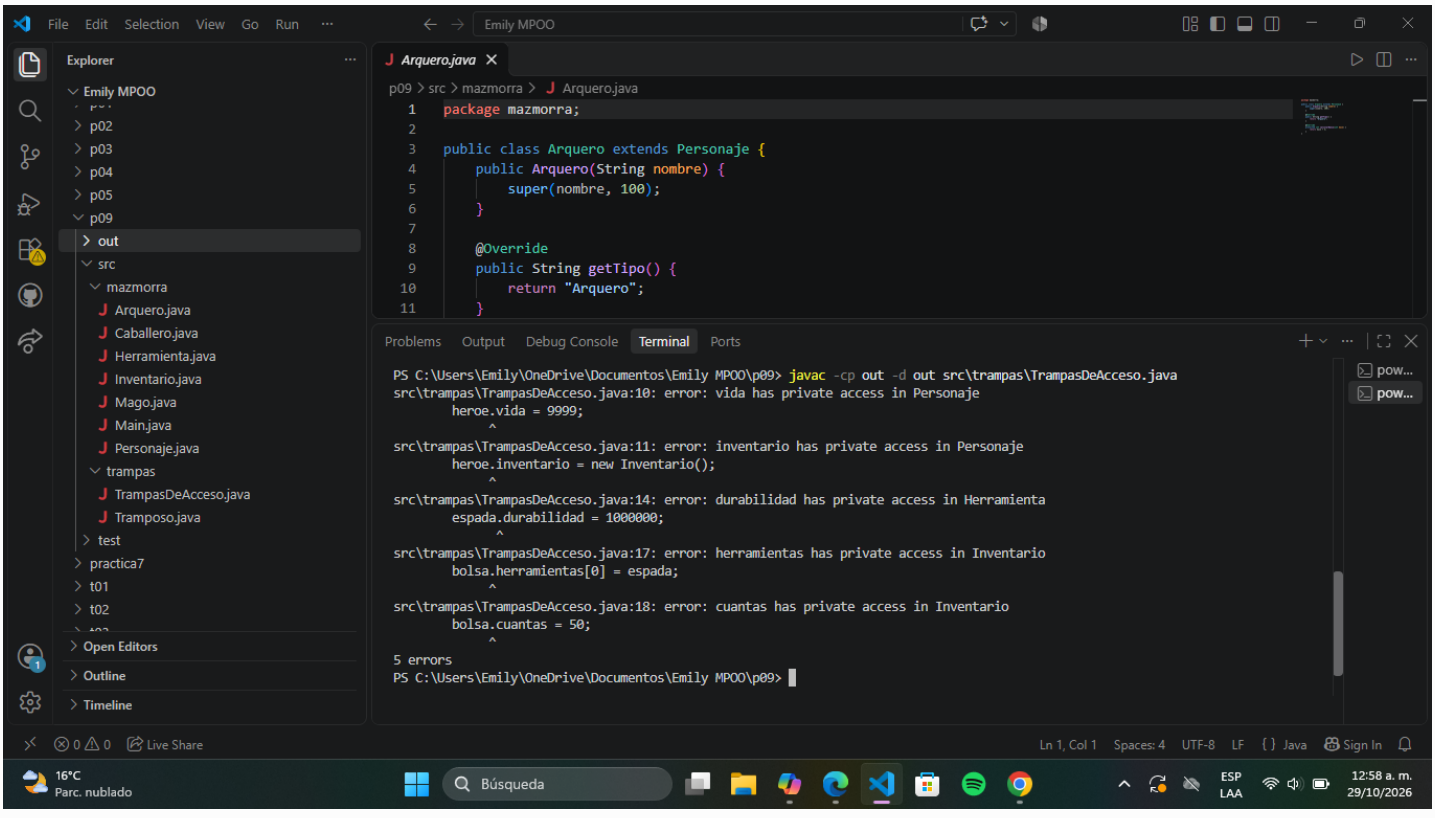



*Figura 3. Errores de acceso a atributos privados.*

También se utilizaron pruebas con JUnit para comprobar que los métodos funcionaran correctamente. Estas pruebas permiten revisar los resultados automáticamente y comparar lo que se esperaba con lo que realmente hizo el programa. En la ejecución realizada se encontraron diez pruebas y las diez terminaron correctamente. El resultado mostró 10 pruebas exitosas y 0 pruebas fallidas.

Entre las cosas que se comprobaron están la vida inicial de cada personaje, el daño que causan las herramientas, el límite de tres herramientas en el inventario, la curación de un compañero y el hecho de que un personaje muerto no pueda recuperar vida.

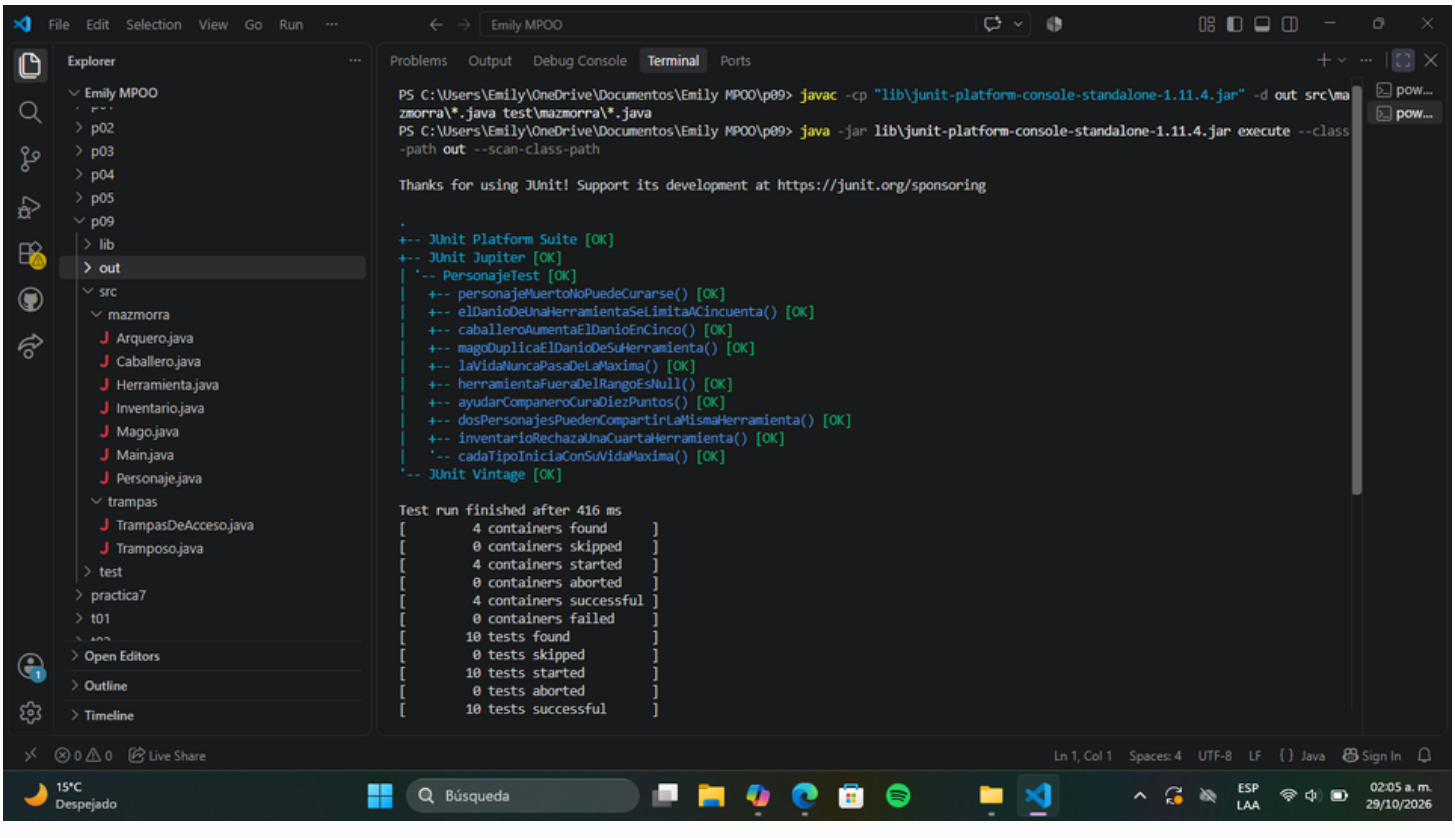



*Figura 4. Resultados de las pruebas con JUnit.*

## Relaciones entre clases

### Herencia

La herencia permite que una clase tome características y métodos de otra. En esta práctica, Caballero, Mago y Arquero heredan de Personaje, porque los tres son tipos de personaje y comparten acciones como atacar, recibir daño y curarse.
La relación se observa en esta línea de Caballero.java:
public class Caballero extends Personaje
Además, en el constructor se utiliza:
super(nombre, 120);
Esta instrucción llama al constructor de Personaje para inicializar los datos comunes del caballero, como su nombre y su vida máxima de 120 puntos. El mago y el arquero hacen lo mismo, pero con 80 y 100 puntos de vida, respectivamente.
En esta relación, la clase hija hereda características de la clase padre. No significa que el personaje hijo sea una parte física del padre, sino que aprovecha lo que ya está definido en Personaje.

### Composición

La composición se presenta entre Personaje e Inventario, porque cada personaje crea su propio inventario dentro de su constructor.
La línea que representa esta relación en Personaje.java es:
this.inventario = new Inventario();
Esto significa que el inventario se crea cuando se construye el personaje. Así, cada personaje tiene su propio inventario y no necesita recibirlo desde fuera.
Según el diagrama, el inventario forma parte del personaje. Por eso se considera composición, a diferencia de las herramientas, que pueden existir independientemente del inventario.

### Agregación

La agregación se presenta entre Inventario y Herramienta. El inventario puede guardar herramientas que fueron creadas previamente fuera de él y que se pasan como parámetro.
Esto se observa en el método de Inventario.java:
public boolean agregar(Herramienta h)
El parámetro h representa la herramienta que se quiere guardar. El inventario no la crea, solamente la recibe y la almacena si todavía tiene espacio.
Una herramienta puede existir antes de entrar al inventario y también puede ser utilizada por más de un personaje. Por ejemplo, dos personajes pueden equipar la misma hacha, por lo que ambos tienen una referencia al mismo objeto.

### Asociación

La asociación se presenta entre dos personajes que pueden ser compañeros. Un personaje puede guardar una referencia a otro para interactuar con él, sin que ninguno forme parte del otro.
Esto se establece mediante el método de Personaje.java:
public void setCompanero(Personaje companero)
El método guarda al compañero que se recibe como parámetro. Después, mediante ayudarCompanero(), un personaje puede curar diez puntos de vida a su compañero, siempre que este siga vivo y se cumplan las reglas de curación.

En esta relación, ambos personajes pueden existir por separado. Si uno deja de ser compañero del otro, eso no significa que el otro personaje deje de existir.

### Huecos

El encapsulamiento sirve para proteger los datos de una clase y evitar que otros objetos los cambien sin respetar las reglas del programa.
Por ejemplo, si cualquier clase pudiera cambiar directamente la vida de un personaje, sería posible ponerle una cantidad de 9999 puntos. Para evitarlo, la clase Personaje no tiene un método setVida(). En su lugar, la vida cambia mediante métodos como recibirDanio() y curar(), que revisan las condiciones antes de modificarla.
Tampoco existe un método getHerramientas() que entregue el arreglo completo del inventario. Si se pudiera acceder directamente a ese arreglo, alguien podría modificar sus posiciones y saltarse el límite de tres herramientas. Por eso se utilizan métodos como getCuantasHerramientas() y getHerramienta(i), que permiten consultar la información sin entregar el arreglo interno.
En la comprobación de TrampasDeAcceso.java, los cinco intentos de acceso directo fueron rechazados por el compilador. Esto demuestra que los atributos privados están protegidos.
En cuanto a las siete trampas de Tramposo.java, las reglas del programa están preparadas para impedir acciones inválidas. Por ejemplo, recibirDanio() ignora cantidades negativas, curar() no permite recuperar vida a un personaje muerto y Inventario.agregar() rechaza una cuarta herramienta cuando el inventario está lleno.

### Una prueba roja

En la guía se muestra una prueba que comprueba si el método recibirDanio() resta correctamente los puntos de vida. En el ejemplo, un mago comienza con 80 puntos de vida y recibe 30 puntos de daño. La prueba esperaba que terminara con 70 puntos, pero el resultado real fue de 50.
El mensaje que aparece es:
expected: <70> but was: <50>
El número 70 representa el resultado que esperaba la prueba, mientras que 50 es el resultado que realmente obtuvo del programa. Como el mago empieza con 80 puntos de vida y recibe 30 de daño, el cálculo correcto es 80 menos 30, que da 50.
Por lo tanto, en este ejemplo el error estaba en la prueba y no en el código. La prueba esperaba una cantidad incorrecta de vida. Esto nos muestra que cuando una prueba aparece en rojo, es necesario revisar tanto lo que se esperaba como el resultado real antes de decidir dónde está el error.

## 7. Conclusión

Con esta práctica comprendí por qué es importante proteger los datos de una clase y no permitir que cualquier objeto los cambie directamente. También aprendí cómo se relacionan las clases mediante herencia, composición, agregación y asociación, y cómo cada relación cumple una función diferente dentro del programa.
Las comprobaciones permitieron revisar que los atributos privados no pudieran modificarse desde una clase externa y que las diez pruebas de JUnit terminaran correctamente. En general, la práctica me ayudó a entender que a parte de que un programa funcione, también debe cumplir con las reglas que se establecieron para cada objeto.